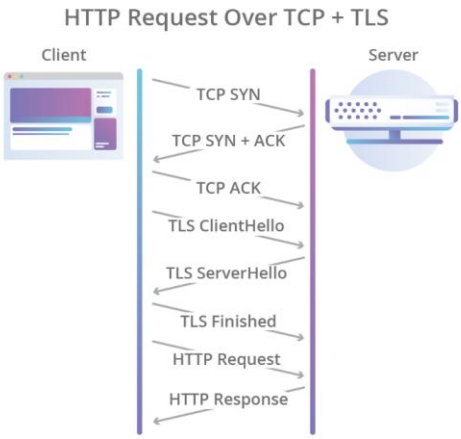
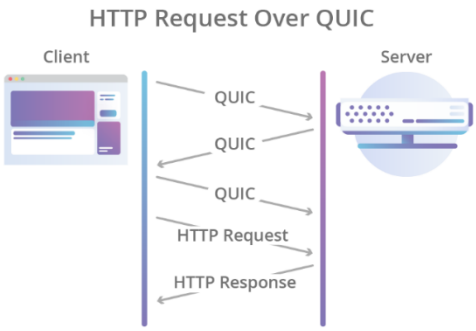
QUIC – Technical Overview
HPE CPP Week-1 Report

Name: Vansh Chani

QUIC (Quick UDP Internet Connections) is a new encrypted-by-default Internet transport protocol, that provides a few improvements designed to accelerate HTTP traffic as well as make it more secure, with the intended goal of eventually replacing TCP and TLS on the web.

The initial QUIC handshake combines the typical three-way handshake that you get with TCP, with the TLS 1.3 handshake, which provides authentication of the end-points as well as negotiation of cryptographic parameters. QUIC replaces the TLS record layer with its own framing format, while keeping the same TLS handshake messages.





Design of QUIC

QUIC is built on top of UDP (User Datagram Protocol) instead of TCP.

Why UDP?

1. Avoids Kernel Restrictions

TCP is implemented inside the operating system kernel.
This means:

- Updating TCP requires OS updates.
- Innovation is slow.
- Developers have limited flexibility.

Since UDP is minimal and simple, QUIC can implement its own transport logic in user space, bypassing kernel-level constraints.

2. Can Be Implemented in User Space

Because UDP just sends raw datagrams:

- QUIC runs entirely in application layer (e.g., inside Chrome, servers like nginx).
- Developers can push updates without waiting for OS upgrades.
- Faster experimentation and deployment.

This is one of the biggest reasons why companies like Google were able to evolve QUIC quickly.

Issues with UDP and how QUIC manages those:

UDP does not guarantee:

- Packet delivery
- Order
- Duplicate prevention

QUIC adds:

- Packet numbering
- Acknowledgment (ACK) frames
- Retransmission of lost packets
- Loss detection mechanisms

Built-in Encryption:

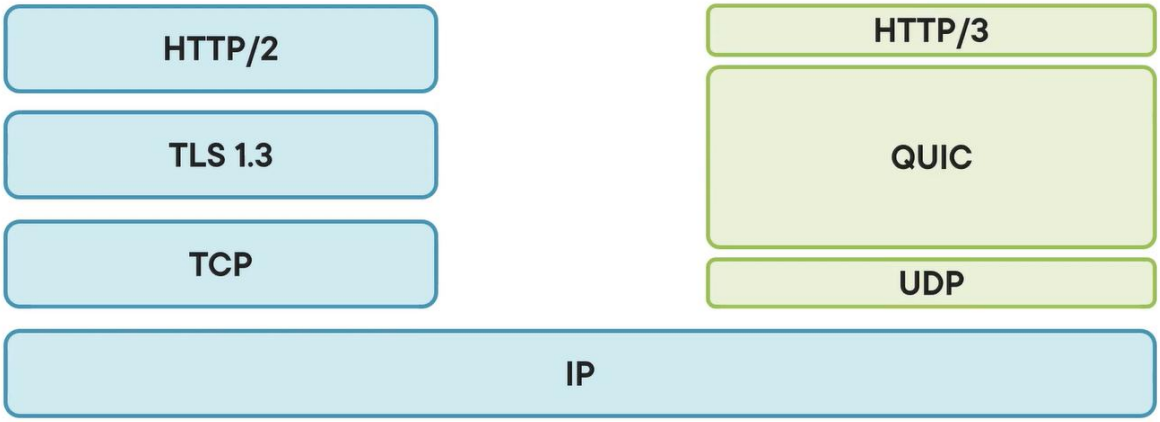
QUIC integrates **TLS 1.3** directly.

There is:

- No unencrypted QUIC
- No separate TLS handshake

Encryption is mandatory.

The QUIC Protocol Stack

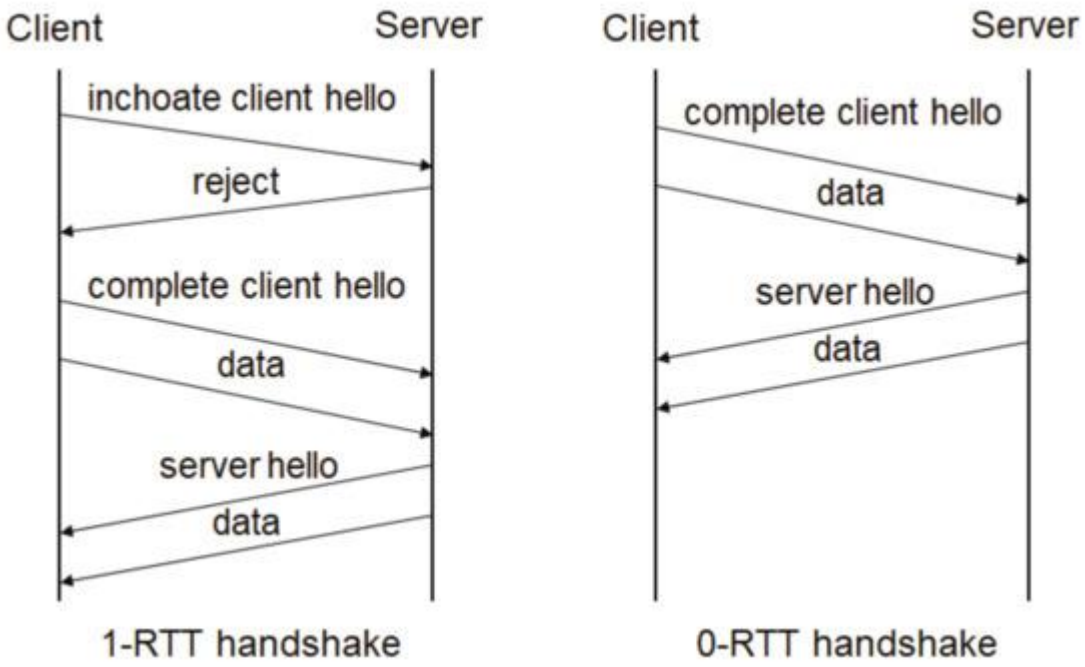


The left stack shows the traditional web setup: HTTP/2 runs on top of TLS 1.3 for security, which runs on TCP for reliable data delivery. TCP ensures packets arrive in order and handles congestion control. All of this sits on top of IP, which routes data across networks.

The right stack shows the modern approach with HTTP/3. Instead of TCP, it uses QUIC, which runs over UDP. QUIC combines security (like TLS), reliability, and congestion control into one protocol.

The main difference is that HTTP/3 replaces TCP + TLS with QUIC over UDP. This allows faster connections, better handling of multiple data streams, and improved performance.

The QUIC Handshake



1-RTT handshake → normal secure handshake

0-RTT handshake → client sends data immediately using previously saved session info

1-RTT:

If no previous session exists:

Client → Server:

- ClientHello (TLS)
- Transport parameters

Server → Client:

- ServerHello
- Keys
- Encrypted response

Data can start flowing after 1 RTT.

0-RTT:

If client has connected before:

Client immediately sends:

- Encrypted application data
- Using stored session keys

No waiting.

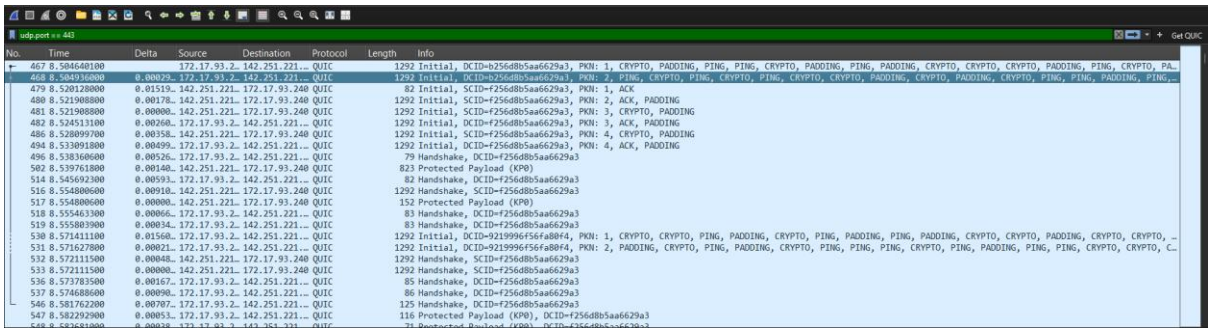
This drastically improves page load time.

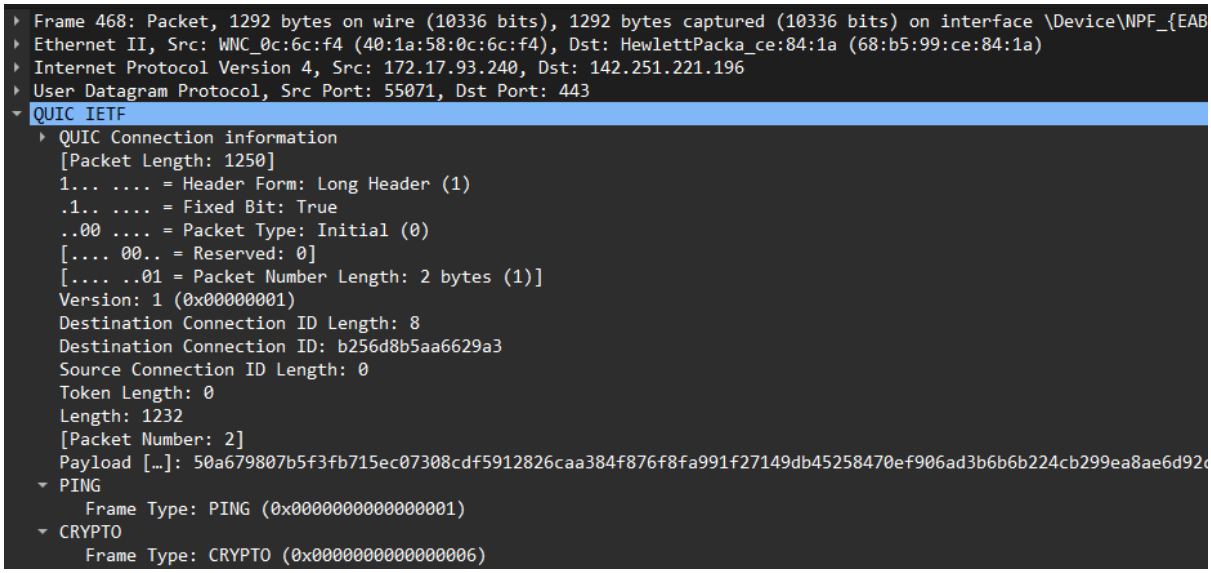
How it happens:

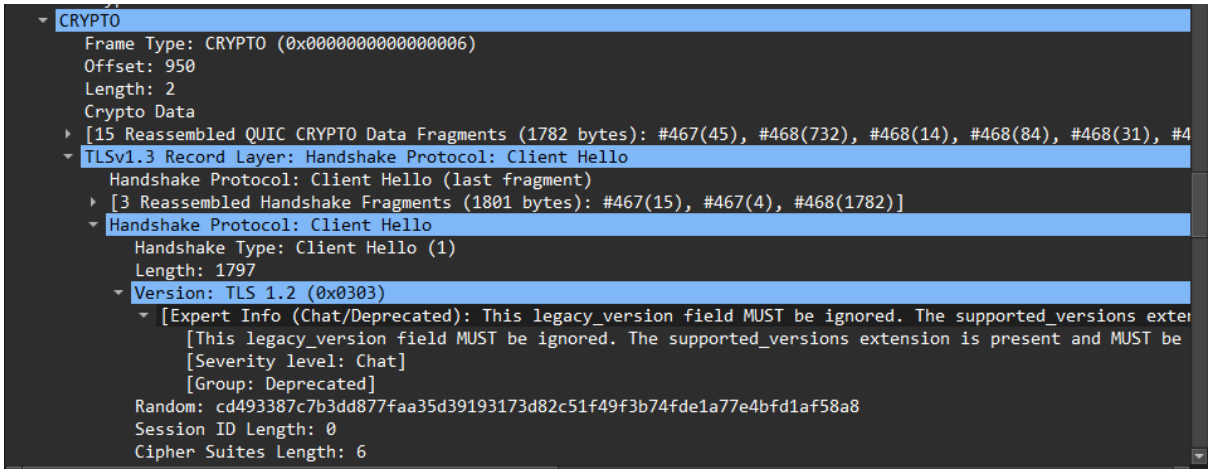
Client first tries 0-RTT but if it fails then server sends "reject" and then it falls back to 1-RTT. If the connection was recent then the parameters match and the server accepts the data sent with the ClientHello itself.

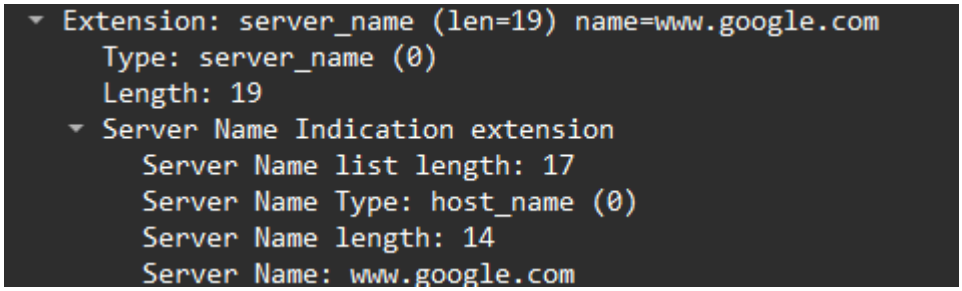
Capturing QUIC with Wire shark:

1. Open Wireshark
2. Select your active network interface (Wi-Fi / Ethernet)
3. Click Start Capture
4. Apply filter udp.port == 443 or quic
5. To generate QUIC traffic visit https://cloudflare-quic.com









Some Distinguishing Features between QUIC traffic and UDP Noise

1. Most QUIC traffic (especially HTTP/3 over QUIC) runs on: **UDP port 443** So if you see: UDP → Port 443 It's likely QUIC.

2. Even though QUIC encrypts most of its payload, its **header format is structured**.
    a. QUIC packets always contain:
    b. Header Form bit (Long or Short)
    c. Fixed bit (The second-most-significant bit of the first byte is always set to 1.)
    d. Version field (in long headers)
    e. Connection ID
    f. Packet number (encrypted but format predictable)

3. Long Header During Handshake:

    During connection setup, QUIC uses a **Long Header** that contains:

    a. Version number (e.g., 0x00000001)
    b. Destination Connection ID
    c. Source Connection ID

    If the Version field matches known QUIC versions → strong indicator it's QUIC.

4. Identification based on Traffic Behaviour:
    QUIC has recognizable traffic behaviour:
    - Initial packets ~1200 bytes (minimum size requirement)
    - TLS 1.3 handshake inside
    - Multiple streams
    - Encrypted frames
    - Regular ACK frames

Main QoS and Policing Methods Used with QUIC

1. DSCP (DiffServ)

Packets can be marked with priority values in the IP header. Routers then prioritize traffic based on those markings.

2. Rate Limiting and Policing

Token bucket or leaky bucket algorithms limit bandwidth per:

- User
- IP address
- Port (commonly UDP 443)
- Flow (5-tuple)

3. Flow-Based Queuing

Routers apply fairness algorithms (e.g., WFQ, FQ) using the 5-tuple:

- Source IP
- Destination IP
- Source Port
- Destination Port
- Protocol (UDP)

Each QUIC connection appears as one UDP flow.

4. Active Queue Management (AQM)

Mechanisms like RED or CoDel manage congestion and reduce latency. These work well because QUIC responds to congestion signals.

5. ECN (Explicit Congestion Notification)

QUIC supports ECN, allowing networks to signal congestion without dropping packets.

6. DPI-Based Policies (Limited)

During handshake, some metadata (like server name) may be visible. Networks can classify traffic at that stage, but visibility is decreasing due to encryption improvements.

7. Blocking or Throttling UDP 443

Some networks block or limit UDP 443 to control QUIC traffic, forcing fallback to TCP.

Summary Table

Method	Works for QUIC?	Granularity
DSCP	Yes	Per packet
Token bucket policing	Yes	Per flow / per user
WFQ / FQ	Yes	Per 5-tuple flow
DPI	Limited	Domain-level
ECN	Yes	Congestion-based
Per-stream QoS	No (not visible to network)	Internal only